

Daily Paper Review: Escaping the Self-Confirmation Trap — An Execute-Distill-Verify Paradigm for Agentic Experience Learning

Internbot — Automated Research Summary

June 25, 2026

1 Paper Summary

1.1 Problem and Motivation

Single-agent experience learning systems suffer from a structural failure mode that Zhu et al. [1] term the *Self-Confirmation Trap*: when the same agent both executes tasks and evaluates the resulting trajectories, the agent’s systematic biases apply equally to both generation and evaluation. If a flawed understanding led to an incorrect trajectory, that same flawed understanding causes the agent to *endorse* the trajectory as correct. Formally, letting π_θ be the agent’s policy, τ the trajectory, $c(\tau) \in \{0, 1\}$ the objective correctness, and $v_{\pi_\theta}(\tau)$ the self-evaluation, the core problem is that $P(v_{\pi_\theta}(\tau) = 1 \mid c(\tau) = 0)$ is substantially elevated when the same policy generates and evaluates.

Wrong-but-self-consistent trajectories are endorsed, written into memory, and retrieved for future tasks, creating a *progressive error amplification loop*. The paper provides striking evidence: (1) injecting 10% erroneous-but-coherent experiences drops the ReasoningBank baseline from 82.5 to 77.2 Pass@1; (2) adding explicit self-verification to a single agent actually *hurts* ($83.3 \rightarrow 83.2$), confirming that a model checking its own homework with the same flawed reasoning provides no benefit; (3) the single-agent baseline’s performance *degrades* as more memories are retrieved (from 0.825 at 1 memory to 0.793 at 3), indicating that additional memories are contaminated and act as distractors.

1.2 Method

Execute: Heterogeneous Parallel Trajectory Generation. A heterogeneous agent pool $\mathcal{A} = \{A_1, \dots, A_K\}$ (different foundation models) is maintained. For each task q , a randomly sampled subset $\mathcal{A}_{\text{exec}}$ independently generates trajectories: $\mathcal{T}(q) = \{\tau_i \mid \tau_i \sim \pi_i(\cdot \mid q), A_i \in \mathcal{A}_{\text{exec}}\}$, with $N = 2$ models each producing $M = 4$ candidates. Heterogeneity is the key—different models expose different success/failure patterns, providing richer contrastive evidence. Models include Mimo-V2-Flash, GLM-4.7-FP8, and MiniMax-M2.1, with high temperature (0.3–1.0) for diverse exploration.

Distill: Third-Party Contrastive Experience Distillation. A designated *third-party* distillation agent $A_{\text{distill}} \in \mathcal{A}$, distinct from the executors, performs cross-trajectory comparison to produce candidate experiences $\mathcal{E}_{\text{cand}} = \{e_1, \dots, e_m\}$. Rather than summarizing any single trajectory, the distiller identifies patterns from *differences between trajectories*—where they agree, disagree, and what emerges from the contrasts. Strict constraints enforce evidence-based extraction: no hallucinations, no generic platitudes, and each experience follows a “Context $\rightarrow$ Risk $\rightarrow$ Correct Action” structure. The third-party role is critical: the distiller has no “skin in

the game”—it did not execute the task, so it has no motivation to rationalize any particular trajectory. This breaks the executor-evaluator coupling that causes the self-confirmation trap.

Verify: Consensus-Based Experience Validation. The verification group is the original execution group: $\mathcal{A}_{\text{verify}} = \mathcal{A}_{\text{exec}}$. Each executor provides a binary judgment $V_j(e) \in \{0, 1\}$ for each candidate experience, operating as a “strict memory audit expert” with a *default-reject mindset*. Candidate experiences must simultaneously pass groundedness, specificity, high utility, and zero redundancy gates.

A strict three-tier storage policy is applied:

- **Unanimous approval** $\rightarrow$ Shared memory bank (generally reusable)
- **Partial approval** $\rightarrow$ Private memory bank of approving agent(s)
- **No approval** $\rightarrow$ Discarded

Why Consensus Verification Works. In the single-agent case, $P(v(e) = 1 \mid e \text{ is flawed})$ is elevated because execution and evaluation biases are correlated. In EDV, flawed experience passing verification requires that *multiple independent evaluators with different biases* all fail to detect the flaw: $P(V_1(e) = 1 \wedge V_2(e) = 1 \mid e \text{ is flawed})$, which is much smaller than any single $P(V_j(e) = 1 \mid e \text{ is flawed})$ assuming partially independent errors across heterogeneous models. The ablation confirms this progression: single-agent self-verification 83.2, independent verifier 84.5, multi-agent self-distillation 85.9, third-party distillation alone 87.1, full EDV **88.6**.

Ability Matrix and Inference. A lightweight component records which agents handle which task types best, enabling intelligent task routing at inference time. Verified memories are retrieved via embedding similarity and prepended to the agent’s prompt.

1.3 Results

Benchmarks. On τ^2 -bench (real-world issue resolution across AIRLINE, RETAIL, TELECOM): EDV achieves **86.6** avg. Pass@1 (vs. 83.5 best multi-agent baseline Router, 81.9 single-agent ReasoningBank). On MMTB (multi-tool task execution): **58.10** overall (vs. 55.96 Router, 53.67 ReasoningBank). On Mind2Web (web navigation): consistently top across all three generalization settings (Cross-Task SSR 43.17 vs. 40.28 Router).

Memory Quality (Human Audit, 5-point scale). EDV dramatically improves memory quality over single-agent ReasoningBank: Groundedness/Correctness 4.41 (vs. 3.72), Actionability 4.32 (vs. 3.58), Specificity 4.27 (vs. 3.64), Noise/Hallucination 0.63 (vs. 1.21, lower is better), Potential Harm 0.51 (vs. 1.08).

Scaling Behavior. EDV’s performance monotonically improves as more memories are retrieved, while the single-agent baseline degrades—the clearest signature that EDV produces higher-quality, non-contaminated memory.

Efficiency. EDV reduces average inference token consumption by **24.5%** vs. ReasoningBank on RETAIL, because higher-quality memory helps agents reach correct solutions faster.

1.4 Limitations

Consensus bias: If all heterogeneous agents share a common blind spot (misunderstand the same concept), the consensus mechanism validates noise. Heterogeneity only protects against *model-specific* biases, not universal ones.

Low-quality agent interference: A significantly underperforming agent can obstruct consensus by blocking valid experiences from achieving unanimous approval or contaminating the distillation process.

Attribution difficulty: The multi-turn interplay between distillers, executors, and verifiers makes root-cause debugging significantly harder than in linear single-agent systems.

Computational overhead: The Execute stage requires running K heterogeneous models in parallel. While offline and parallelizable, total compute is roughly $K \times$ single-agent.

Small pool: Only 3 models tested. Scaling behavior from 3 to ~ 10 participants is unexplored.

1.5 Relevance to Our Research

EDV addresses a failure mode that permeates our entire T3 review corpus. Our Continual Experience Internalization (CEI) review [2] analyzed how agents internalize experience over time, but assumed the experience itself is reliable. EvoArena [3] identified the “state collapse” problem when environment dynamics invalidate cached knowledge, and EvoMem’s git-like patches track *what changed*—but neither addresses whether the experience being cached was correct in the first place. EDV complements both by ensuring that only verified, high-quality experience enters the memory system.

The self-confirmation trap is deeply connected to TRIAGE’s [4] risk polarization diagnosis. TRIAGE showed that one-sided reasoning collapses LLM confidence into near-deterministic outputs (99.98% predicted-class probability), solved by *dialectical reasoning*—separately examining evidence for each outcome. EDV generalizes this insight from reasoning structure to the entire experience learning loop: the Execute-Distill-Verify decomposition is analogous to TRIAGE’s thesis-antithesis-synthesis, where heterogeneous execution provides diverse perspectives (thesis/antithesis) and consensus verification synthesizes the best experience.

The finding that self-verification *hurts* ($83.3 \rightarrow 83.2$) directly parallels our self-distillation degradation review [5], which showed that self-distillation can degrade reasoning when the model reinforces its own biases. EDV and TRIAGE provide complementary solutions: TRIAGE restructures the reasoning process itself, while EDV separates the actors in the learning loop.

EvoEmbedding’s [6] finding that “simple RAG + better embeddings > complex memory systems” assumed high-quality memory contents. EDV’s memory contamination experiment (5.3-point degradation from 10% noisy memory) demonstrates that embedding quality is necessary but not sufficient—memory *content quality* is equally critical. Together, EvoEmbedding and EDV provide a complete solution: EvoEmbedding ensures the right memory is retrieved; EDV ensures the retrieved memory is correct.

2 Follow-up Research Directions

2.1 Direction 1: Dialectical Experience Distillation with Outcome-Conditioned Rationales

Gap. EDV’s third-party distillation performs cross-trajectory comparison but does not structurally guarantee balanced consideration of success and failure patterns. The distiller may still exhibit one-sided attention—focusing primarily on what worked in successful trajectories while underweighting informative failure patterns. TRIAGE [4] showed that dialectical reasoning

(separately examining evidence for each outcome) dramatically improves calibration; EDV’s distillation lacks this structural guarantee.

Approach. Replace EDV’s free-form distillation with TRIAGE-style dialectical distillation. For each task, the distiller produces two outcome-conditioned analyses: (a) a rationale explaining why the *best* trajectory succeeded, identifying specific decisions that contributed to success, and (b) a rationale explaining why the *worst* trajectory failed, identifying specific decisions that led to failure. The distilled experience must synthesize both rationales into a contrastive lesson: “In context C , action A_{good} succeeds because R_+ , while action A_{bad} fails because R_- .” This forces the distiller to examine both sides of the evidence before extracting a lesson, preventing one-sided distillation bias. The same dialectical structure also improves verification quality: verifiers evaluate whether both the success and failure attributions are grounded in trajectory evidence.

Feasibility. The dialectical prompt structure is a straightforward modification to EDV’s distillation template. TRIAGE demonstrated that dialectical reasoning improves calibration with minimal prompt engineering. Validation on τ^2 -bench comparing standard EDV distillation vs. dialectical distillation, measuring memory quality (human audit scores) and downstream Pass@1. The key metric is whether dialectical distillation reduces the Noise/Hallucination score below EDV’s already-low 0.63.

2.2 Direction 2: Branching Score-Guided Experience Extraction from Agent Trajectories

Gap. EDV extracts 1–3 high-value memory items per task through holistic trajectory comparison, but does not identify *which specific decision points within a trajectory* are most consequential. Our APPO review [7] introduced the Branching Score ($BS = Z(\Omega) \cdot Z(\text{Entropy})$) to identify latent decision points in agentic RL trajectories—positions where the policy’s downstream behavior diverges most. Applying BS to EDV’s trajectories could pinpoint the exact moments where successful and failed trajectories diverge, enabling more precise experience extraction.

Approach. During EDV’s Execute stage, compute BS for each token in each trajectory. Identify the top- k BS positions per trajectory—these are the “critical decision points” where the agent’s choices had the largest downstream impact. During the Distill stage, present the distiller not with full trajectories but with *aligned critical segments*: for each high-BS position, show how different executors handled the same decision context and what outcomes followed. This transforms distillation from holistic trajectory comparison to *procedure-level* contrastive analysis, enabling extraction of more specific, actionable experiences (“At this exact decision point, the agent should choose A over B because...”) rather than generic task-level lessons.

Feasibility. BS computation requires forward passes through the agent’s policy at each trajectory position, which EDV already has (the trajectories are generated by the executors). Aligning high-BS positions across trajectories requires finding corresponding decision contexts, achievable via embedding similarity of the trajectory prefix at each BS position. Validation: compare standard EDV vs. BS-guided EDV on MMTB and τ^2 -bench, measuring experience specificity (human audit) and downstream efficiency (token consumption reduction).

2.3 Direction 3: Self-Confirmation Detection via EvoEmbedding Trajectory Divergence

Gap. EDV prevents self-confirmation through structural separation (third-party distillation + consensus verification), but cannot *detect* self-confirmation in existing single-agent systems or

flag when consensus verification itself is failing (the consensus bias limitation). No mechanism currently exists to *measure* the degree of self-confirmation in an agent’s memory—to quantify whether stored experiences reflect genuine environmental knowledge or self-reinforced biases.

Approach. Use EvoEmbedding’s [6] evolving representations to detect self-confirmation. The key insight: if an agent’s memories are genuinely informative, their EvoEmbedding representations should *diverge* from the agent’s own trajectory embeddings (they add new information the agent didn’t already have). If memories are self-confirming, their EvoEmbeddings should be *highly similar* to the generating trajectory’s embeddings (they merely restate what the agent already believes). Define a Self-Confirmation Index (SCI): $\text{SCI}(e, \tau) = \cos(v_e^{\text{evo}}, v_\tau^{\text{evo}})$, where v_e^{evo} and v_τ^{evo} are EvoEmbedding representations of the experience and generating trajectory conditioned on the same memory state M_t . High SCI indicates self-confirmation (the experience is redundant with the trajectory that produced it); low SCI indicates genuine new knowledge. Use SCI as an additional verification gate: reject experiences with SCI above a threshold η , even if they pass consensus. This addresses EDV’s consensus bias limitation by catching cases where all agents share a common blind spot—their shared bias produces self-confirming experiences with high SCI.

Feasibility. EvoEmbedding-4B is plug-and-play (demonstrated +19–21% on existing memory systems). SCI computation requires two EvoEmbedding forward passes per experience (one for the experience text, one for the generating trajectory). The threshold η can be calibrated on a held-out set with known correct/incorrect experiences. Validation: inject known self-confirming experiences (from EDV’s contamination experiment) and measure SCI’s ability to detect them, comparing detection rate against consensus verification alone.

3 Conclusion

EDV identifies and addresses the Self-Confirmation Trap—a structural failure mode where single-agent experience learning creates progressive error amplification loops because the same biases that produce errors also prevent their detection. The Execute-Distill-Verify decomposition breaks this cycle through three structurally separated stages: heterogeneous parallel execution provides diverse trajectory evidence, third-party contrastive distillation removes executor bias from experience extraction, and consensus-based verification with a default-reject policy gates memory quality through multi-perspective validation. The finding that self-verification *hurts* performance ($83.3 \rightarrow 83.2$) while the full EDV pipeline improves it to 88.6 provides compelling evidence that the separation of roles matters as much as the quality of any individual component. For our research program, EDV establishes that memory *content quality* is a prerequisite for effective agentic memory systems—complementing EvoEmbedding’s contribution to memory *retrieval quality*—and the self-confirmation diagnosis generalizes TRIAGE’s one-sided reasoning critique from individual inference to the entire agent learning lifecycle.

References

- [1] Zhu, S., Qi, Y., Wang, Y., Li, J., Song, C., Shi, Y., Miao, Y., Gao, H., and Zhang, K. “Escaping the Self-Confirmation Trap: An Execute-Distill-Verify Paradigm for Agentic Experience Learning.” *arXiv preprint arXiv:2606.24428*, 2026.
- [2] Liu, J., et al. “Rethinking Continual Experience Internalization for Self-Evolving LLM Agents.” *arXiv preprint arXiv:2606.04703*, 2026.
- [3] Xu, J., et al. “EvoArena: Tracking Memory Evolution for Robust LLM Agents in Dynamic Environments.” *arXiv preprint arXiv:2606.13681*, 2026.

- [4] Jang, H., et al. “TRIAGE: Dialectical Reasoning for Explainable Risk Prediction on Irregularly Sampled Medical Time Series with LLMs.” *arXiv preprint arXiv:2606.09030*, 2026.
- [5] Wang, Z., et al. “Why Does Self-Distillation (Sometimes) Degrade the Reasoning Capability of LLMs?” *arXiv preprint arXiv:2603.24472*, 2026.
- [6] Nie, C., et al. “EvoEmbedding: Evolvable Representations for Long-Context Retrieval and Agentic Memory.” *arXiv preprint arXiv:2606.21649*, 2026.
- [7] Wang, X., et al. “APPO: Agentic Procedural Policy Optimization.” *arXiv preprint arXiv:2606.12384*, 2026.
- [8] Chen, X., et al. “SkillOS: Learning Skill Curation for Self-Evolving Agents.” *arXiv preprint arXiv:2605.06614*, 2026.
- [9] Li, Y., et al. “SkillOpt: Executive Strategy for Self-Evolving Agent Skills.” *arXiv preprint arXiv:2605.23904*, 2026.